ACS-3913-770
Assignment 2
Due by Monday, October 26 at 11:59 pm

Read and review the Git Fundamentals document on Nexus. All code, documents, and diagrams will be submitted through Nexus.

Download the Assignment 2 starter code. Follow the Workflow section of the Git Fundamentals document to initialize the assignment folder as a local Git repository and create your initial commit.

Notes:

- *Create diagrams with any software drawing tool and save them as PDF files.*
- ***You are expected to use Git regularly throughout the assignment, adding and committing your work as you progress***
- *All files created for this assignment, including code, documents, and diagrams, must be added and committed to your Git repository.*
- *Your Git repository and commit history must be included with your final submission. Further submission instructions are provided at the end of the assignment.*

PART A

1. **State Pattern**

   Consider the states in a special version of the game *Super Mario Bros*.

   The game starts with our hero in his default state (Mario). When Mario acquires a mushroom, he turns into Super Mario. If Super Mario encounters an enemy and gets hit, he goes back to being Mario, and if Mario encounters an enemy and gets hit, he is defeated. If a star is acquired, then he is invincible, so when he encounters an enemy, he is not affected. After 20 seconds, Mario loses his invincibility and returns back to his previous state. Mario and Super Mario can also be defeated if he falls into a void, even if he is invincible.

   When our hero is defeated, if he has at least one life left, he respawns into Mario's default state; otherwise, the game is over.

   a) Show with a statechart diagram the character's gameplay. Show all states, actions/transitions, and guards (as necessary).
   b) Show the class diagram that results when you represent the game using the State design pattern. Include methods and important fields to illustrate the pattern.

c) Use a sequence diagram to show the messages sent if Mario acquires a mushroom, takes a hit from an enemy, acquires a star, and encounters an enemy, in that specified order.

2. Façade Pattern

In this scenario, a new version of the Super Mario World Game Engine includes several complex subsystems responsible for saving player progress. These subsystems include:

- `PlayerData` – Stores and retrieves the player's stats, inventory, and achievements.
- `GameWorld` – Tracks the current level, enemy positions, and collected items.
- `CheckpointSystem` – Marks and restores checkpoints.
- `NotificationService` – Displays on-screen messages (e.g., "Checkpoint Saved!").

Saving progress currently requires multiple steps that the game developer must call manually. To simplify this, design a `GameSaveFacade` class that provides a single method, `saveGame()`, which coordinates all of the following: (1) Retrieve and save player data (2) Record the current level and checkpoint (3) Save world state (4) Notify the player that the game was saved successfully

a. Draw a class diagram showing how `GameSaveFacade` interacts with the subsystems.
b. Draw a sequence diagram illustrating the flow of saving a game using the `GameSaveFacade`.

PART B

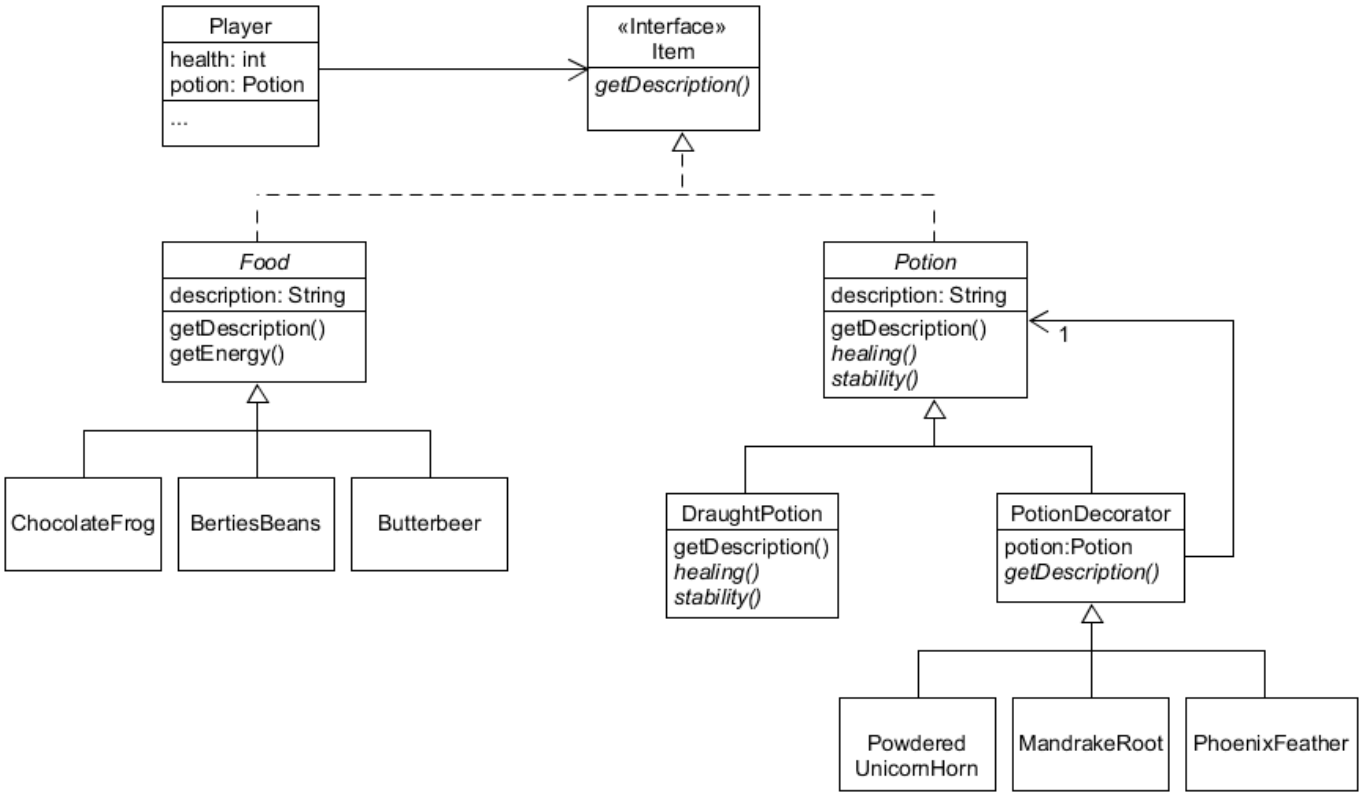
In this part, you will design and model the brewing and enhancement system of a small potion-making simulation game.

In the wizarding world, players can pick up food items, potions, or enhancements. The player is an apprentice potion-maker who brews a basic potion (Base Draught) and then enhances it with magical ingredients. Each enhancer improves the potion's *healing* and *stability* attributes.

	healing	stability
Base potion:		
Base Draught	1	1
Enhancements:		
Powdered Unicorn horn	+2	+1
Mandrake Root	+1	+2
Phoenix Feather	+3	+3

	energy
Food items:	
Chocolate Frog	+1
Bertie Botts' Every Flavour Beans	+2
Butterbeer	+3

Use the following class diagram to implement the Food and Potion items.



1. **Decorator Pattern**

   a. Draw a **sequence diagram** for displaying potion stats when a Base Draught receives enhancers in order:
   Mandrake Root → Powdered Unicorn Horn → Mandrake Root → Phoenix Feather.
   Show the calls that produce the final description and computed stats.

   b. Implement the design.
      i. Complete the code with the Player class (with any other required fields/methods), and required classes of the Items hierarchy.
      - Starter code provided: `Item` interface, abstract `Food` class, and concrete `Food` items

ii. Program a simulation of your system in use. Create a driver class named `As2PartBDriver` that illustrates the upgrade system in "gameplay".

- The game simulation starts with the player with 1 health and no potion (potion is null, but can be the first item that the player picks up).
- Present random items (food and enhancements) to the player
  - An additional "exploding cauldron" item may be presented to the player: the game ends if the player picks it up
- The player will decide if they want to use each item
- Update and display the health and potion stats as applicable

Notes:

- Potions can have unlimited enhancements and stats
- Consider using JOptionPane for player input using showConfirmDialog e.g.

```
result = JOptionPane.showConfirmDialog(null, enhancement + " : pick up?",null, JOptionPane.YES_NO_OPTION);
```

***** BE SURE TO COMMIT ALL YOUR CHANGES BEFORE MOVING ON TO PART 2 *****

2. **Adapter Pattern**

Add a new feature to your system: the ability to use a Food item as a potion! When the player first encounters the Butterbeer, the system will give them the option to equip it as a potion called *Elixir of Froth*. From there, the player can add enhancements to their new Butterbeer-based potion.

This new feature will use the Adapter pattern, where the adaptee is the `Food` object.

a. Add a new class `FoodAdapter` to the class diagram that implements the methods of the `Potion` class with the intent for the food item (Butterbeer) to be used as a concrete potion (you can simply draw a segment of the class diagram, that represents this pattern)

b. Implement your new feature.
   - Add the applicable methods to the `FoodAdapter` where the food item's healing corresponds to the food's energy, and the stability is 0.
   - In the `Player` class, include the new field <*SpecialPotion*> e.g. `ButterbeerPotion` of type `Potion` and any applicable methods required for your simulation.

c. Update the game simulation to include your new feature. When the player is presented with a special item, they are given the choice to add it as a new potion. After it is assigned as the *SpecialPotion*, the player can choose to upgrade it with enhancements found in the world.

After equipping it as a potion, any other *special item* (e.g. Butterbeer) found in the world will only be consumed for energy.

Notes:

- Consider using JOptionPane for choosing between the two potions with showOptionDialog e.g.

```
String[] buttons = {"draught", "butterbeer"};
JOptionPane.showOptionDialog(null, "Select the potion to enhance:
", null,JOptionPane.DEFAULT_OPTION,
JOptionPane.INFORMATION_MESSAGE, null, buttons, buttons[1]);
```

- Test to make sure that the getDescription() provides a meaningful string of the layers of decorators e.g. `Mandrake Root, Powdered Unicorn Horn, Elixir of Froth`

  See Sample game sim output below

Before submitting your assignment, make sure that all files and changes have been committed to your Git repository.

Check the status of your repository and review your commit history as described in the Git Fundamentals document.

Zip the entire assignment folder, including the .git folder, and submit the ZIP file through Nexus.

IMPORTANT: The .git folder contains your Git repository and commit history and must be included in your submission.

Sample game sim output

```
****************************************
*** <another awesome game name here> ***
****************************************
```

Base draught: pick up?
[yes] [no]

```
[New potion: base draught]

Base Draught  |  healing: 1  |  stability: 1
```

Bertie's beans: pick up?
[yes] [no]

```
[Ate the beans]

Health: 3 HP
```

Chocolate frog: pick up?
[yes] [no]

```
[Ate the frog]

Health: 4 HP
```

Mandrake root: pick up?
[yes] [no]

```
[Added enhancement: mandrake root]

Base Draught  |  healing: 2  |  stability: 3
```

Butterbeer: pick up?
[yes] [no]

Do you want to use the Butterbeer as a potion?
[yes] [no]

[New potion: Elixir of Froth]

Elixir of Froth | healing: 3 | stability: 0

Chocolate frog: pick up?
[yes] [no]

[Ate the chocolate frog]

Health: 5 HP

Unicorn powder: pick up?
[yes] [no]

Select the potion to enhance:
[base draught] [elixir of froth]

[Added enhancement: unicorn powder]

Elixir of Froth | healing: 5 | stability: 1

Mandrake: pick up?
[yes] [no]

Select the potion to enhance:
[base draught] [elixir of froth]

[Added enhancement: mandrake]

Base Draught | healing: 3 | stability: 5

Butterbeer: pick up?
[yes] [no]

[Ate the butterbeer]

Health: 8 HP

Exploding cauldron: pick up?
[yes] [no]

[Cauldron exploded... Seamus!!!]

*** Game over :(***